Chittagong University of Engineering and Technology

Department of Electronics and Telecommunication Engineering

PROJECT REPORT

Project Title:
InduSense: Smart Industrial Environment Monitoring & Control System.

Course No: **ETE 416**

Course Title: **IOT and Industrial Automation Sessional**

Date of Submission: **04-04-2026**

Submitted By

Shahriar Kibria Jawwad (2008024)

Taspia Hanif Anna (2008034)

Rudro Roy (2008040)

Yousuf Asgor Mumin (2008046)

Submitted To

Taieba Taher
Assistant Professor
Department of ETE, CUET

Introduction

The rapid advancement of Industry 4.0 and the Industrial Internet of Things (IIoT) has revolutionized traditional manufacturing into smart, data-driven operations. Modern industrial facilities require continuous, real-time monitoring of critical environmental parameters including temperature, humidity, air quality, gas leakage, and fluid levels to ensure worker safety, equipment protection, and regulatory compliance.

InduSense is a comprehensive Smart Industrial Environment Monitoring & Control System designed to address the growing need for affordable, scalable, and reliable industrial safety solutions. The system integrates multiple sensors with an ESP32 microcontroller, implements Real-Time Operating System (RTOS) concepts for reliable multitasking, and provides both local and cloud-based dashboards for real-time monitoring and control. An Adaptive Threshold Engine is incorporated to make fan control context-aware, allowing the temperature trigger point to be updated dynamically from the cloud rather than relying on a hardcoded constant.

Traditional industrial monitoring systems, while reliable, present significant challenges including high costs, complex installation, vendor lock-in, and limited scalability. InduSense bridges this gap by offering a cost-effective alternative that leverages open standards and off-the-shelf components without compromising on industrial-grade reliability.

The system architecture follows a five-layer IoT model: Perception Layer (sensors), Network Layer (WiFi/Bluetooth), Middleware Layer (cloud/fog), Application Layer (dashboards), and Business Layer (alerts and decision-making). This structured approach ensures robustness, scalability, and security across all system components. Figure **??** illustrates this layered architecture.

Motivation

Industrial environments face serious safety challenges due to various hazards such as fire accidents, gas leakage, chemical exposure, and machinery-related risks. Recent industrial incidents are:

- Fire at Chittagong Export Processing Zone (2025)
- Fire at Mirpur Factory (2025)

The major problems identified are:

- Delayed detection of hazardous conditions
- Dependence on manual monitoring
- Lack of real-time alerts and automation

- High risk to human safety and industrial infrastructure

These motivated the development of a smart, automated, and real-time monitoring system that enhances safety and reduces human dependency.

Objectives

The main objectives of this project are:

- To develop an industrial environment monitoring and control system
- To integrate multi-parameter sensing (temperature, humidity, gas, air quality, liquid level)
- To implement RTOS-based multitasking for real-time operation
- To enable local and cloud-based dashboards for monitoring and control
- To incorporate PLC-oriented automation logic
- To apply data analytics concepts for intelligent decision-making
- To implement an Adaptive Threshold Engine for context-aware fan control via cloud-updatable thresholds

Core Components

- ESP32 Microcontroller
- DHT22 (Temperature & Humidity Sensor)
- MQ-135 (Air Quality Sensor)
- MQ-2 (Gas Sensor)
- HC-SR04 (Ultrasonic Sensor)
- Relay Module (Actuator Control)
- OLED Display

Implementation

Hardware Setup

The hardware setup of the system is based on the ESP32 microcontroller, which acts as the central processing unit. Various sensors and actuators are interfaced with the ESP32 as follows:

- **DHT22 Sensor:** Connected to GPIO pin 4 for temperature and humidity measurement.
- **MQ-135 Sensor:** Connected to analog pin 34 for air quality monitoring.
- **MQ-2 Sensor:** Connected to analog pin 35 for gas leakage detection.
- **HC-SR04 Ultrasonic Sensor:** Connected to GPIO pins 5 (TRIG) and 18 (ECHO) for liquid level measurement.
- **Relay Module:** Connected to GPIO pins 26, 27, and 33 to control fan, pump, and gas alarm.
- **Buzzer:** Connected to GPIO pin 25 for alert generation.
- **OLED Display:** Connected via I2C (GPIO 21, 22) for local data visualization.

All components are powered using a stable DC power supply, ensuring reliable operation. Figure 1 shows a photograph of the assembled hardware prototype.

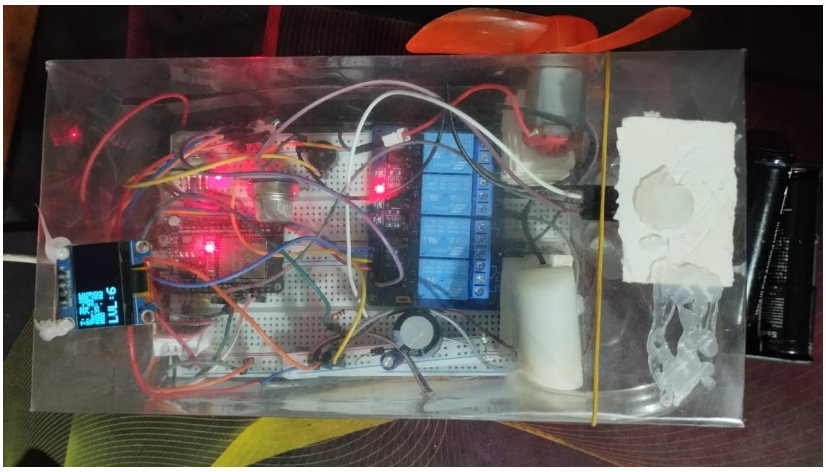


Figure 1. Assembled InduSense hardware prototype.

Software Setup

The software implementation is developed using the Arduino IDE with ESP32 board support. The following libraries are used:

- `WiFi.h` for network connectivity
- `HTTPClient.h` for cloud communication
- `ArduinoJson` for JSON data handling
- `DHT` library for temperature and humidity sensing
- `Adafruit_SSD1306` for OLED display

The system uses FreeRTOS features available in ESP32 for multitasking. Firebase is used as the cloud platform for real-time data storage and remote control. An Adaptive Threshold Engine periodically reads a fan temperature threshold from the `/ml/thresholds/fan` Firebase path, allowing the trigger point to be updated from the cloud without reflashing the device.

ESP32 Code Implementation

The ESP32 firmware is organized into distinct functional blocks, each responsible for a specific aspect of the system: library includes and pin definitions, system initialization, sensor reading helpers, actuator control, Firebase data upload, cloud command polling, the Adaptive Threshold Engine, and the main execution loop. The following sub-sections present each block separately for clarity.

1. Library Includes, Pin Definitions & Global Variables

All required libraries are included at the top of the firmware. Hardware pin assignments and threshold constants are defined using `#define` macros for easy modification. Global variables manage timing intervals, pump state, manual override flags, and the `adaptiveTempThreshold` variable that stores the fan trigger temperature received from Firebase. A dedicated `THRESHOLD_INTERVAL` timer controls how often the threshold is refreshed from the cloud.

Listing 1. Library includes, pin definitions, and global variable declarations

```
// InduSense - ESP32 with Firebase REST + NTP timestamp
// Requires ArduinoJson v6

#include <WiFi.h>
#include <HTTPClient.h>
```

```
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>
#include <time.h>
#include <Wire.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ================= OLED CONFIG =================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ================= DHT22 CONFIG =================
#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// ================= MQ SENSORS =================
#define MQ135_PIN 34
#define MQ2_PIN   35
#define MQ135_THRESHOLD 1000
#define MQ2_THRESHOLD   500

// ================= HC-SR04 CONFIG =================
#define TRIG_PIN 5
#define ECHO_PIN 18
#define PUMP_ON_LEVEL 5
#define PUMP_OFF_LEVEL 2.5

// ================= RELAY CONFIG =================
#define RELAY_FAN 26
#define RELAY_PUMP 27
#define RELAY_GAS 33
#define BUZZER_PIN 25

// ================= NETWORK / FIREBASE =================
const char* ssid = "CUET-Students";
const char* password = "1020304050";
const char* firebaseHost =
  "indusense-9ecf4-default-rtdb.asia-southeast1.firebasedatabase.app";
const char* deviceId = "device1";

// Firebase path for adaptive ML threshold
const char* thresholdPath = "/ml/thresholds/fan";

```

```
// ================= TIMERS =================
unsigned long lastPostMillis = 0;
unsigned long lastPollMillis = 0;
unsigned long lastThresholdMillis = 0;

const unsigned long POST_INTERVAL      = 5000; // upload sensor data every 5s
const unsigned long POLL_INTERVAL      = 2000; // poll manual commands every 2s
const unsigned long THRESHOLD_INTERVAL = 5000; // refresh adaptive threshold every 5s

// ================= STATE =================
bool pumpState = false;

// Adaptive threshold (fallback = 27.0 C if Firebase unreachable)
float adaptiveTempThreshold = 27.0;

// Manual override flags
bool overrideFan = false;    bool overrideFanVal = false;
bool overridePump = false;   bool overridePumpVal = false;
bool overrideBuzzer = false; bool overrideBuzzerVal = false;

WiFiClientSecure client;
HTTPClient http;
```

2. Setup Function – WiFi, NTP & Peripheral Initialization

The `setup()` function runs once on boot. It initializes serial communication, configures all GPIO pins, starts the OLED display, connects to the WiFi network, and synchronizes system time using NTP servers. After connectivity is established, `fetchAdaptiveThreshold()` is called immediately so the correct fan threshold is available before the first control cycle runs. If NTP is unavailable, `millis()` is used as a fallback timestamp.

Listing 2. setup() – peripheral initialization, WiFi connection, NTP sync, and initial threshold fetch

```
void setup() {
  Serial.begin(115200);
  delay(100);

  dht.begin();
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);

  pinMode(TRIG_PIN,   OUTPUT);
  pinMode(ECHO_PIN,   INPUT);
  pinMode(RELAY_FAN,  OUTPUT);
  pinMode(RELAY_PUMP, OUTPUT);
```

```
  pinMode(RELAY_GAS,  OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(RELAY_FAN,  HIGH);  // relays OFF at startup
  digitalWrite(RELAY_PUMP, HIGH);
  digitalWrite(RELAY_GAS,  HIGH);

  Wire.begin(21, 22);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found");
    while(1);
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("ESP32 Monitoring System");
  display.println("Starting...");
  display.display();
  delay(1000);

  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED) {
    delay(300); Serial.print(".");
    if (millis() - wifiStart > 20000) break;
  }
  Serial.println();
  if (WiFi.status() == WL_CONNECTED) {
    Serial.print("WiFi IP: "); Serial.println(WiFi.localIP());
    display.clearDisplay(); display.setCursor(0, 0);
    display.print("WiFi IP: "); display.println(WiFi.localIP());
    display.display();
  } else {
    Serial.println("WiFi failed");
    display.clearDisplay(); display.setCursor(0, 0);
    display.println("WiFi failed");
    display.display();
  }

  // NTP sync (UTC)
  configTime(0, 0, "pool.ntp.org", "time.google.com");
  Serial.print("Waiting for NTP");
  unsigned long tstart = millis();
  while (time(NULL) < 1600000000) {
    delay(500); Serial.print(".");
```

```
    if (millis() - tstart > 8000) break;
  }
  Serial.println();
  if (time(NULL) >= 1600000000) Serial.println("NTP time obtained");
  else Serial.println("NTP not ready -- using millis() as fallback");

  client.setInsecure(); // prototyping only

  // Fetch adaptive threshold immediately at boot
  fetchAdaptiveThreshold();
}
```

3. Sensor Reading Helper Functions

Two helper functions handle sensor data acquisition. `readDistanceCM()` triggers the HC-SR04 ultrasonic sensor and computes the distance in centimetres from the echo pulse duration. `parseJsonBool()` is a utility that safely extracts boolean values from Firebase JSON command payloads, supporting string forms such as `"yes"`, `"true"`, `"1"`, and `"on"`.

Listing 3. Helper functions – ultrasonic distance measurement and JSON boolean parser

```
// ======= Read distance from HC-SR04 =======
long readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) return -1;
  return duration * 0.034 / 2;
}

// ======= Parse boolean value from JSON variant =======
bool parseJsonBool(JsonVariant v, bool &hasValue) {
  if (v.isNull()) { hasValue = false; return false; }
  hasValue = true;
  if (v.is<bool>()) return v.as<bool>();
  const char* s = v.as<const char*>();
  if (!s) return false;
  String S = String(s); S.toLowerCase();
  return (S == "yes" || S == "true" || S == "1" || S == "on");
}
```

4. Actuator Control Logic

The `applyActuators()` function implements the threshold-based automation logic. The key change introduced by the Adaptive Threshold Engine is that the fan now compares the current temperature against `adaptiveTempThreshold` — a runtime variable updated from Firebase — rather than a hardcoded constant. This makes the fan trigger point adjustable without reflashing the firmware. The pump still responds to the ultrasonic liquid-level sensor using hysteresis, and the buzzer and gas relay activate when either MQ sensor detects a hazardous condition. Manual override commands from Firebase take precedence over all automatic logic.

Listing 4. applyActuators() – adaptive threshold-based fan control with manual override support

```
void applyActuators(float temperature, long distance,
                    bool mq135Detected, bool mq2Detected) {
  // Fan -- uses adaptive threshold from Firebase
  if (overrideFan)
    digitalWrite(RELAY_FAN, overrideFanVal ? LOW : HIGH);
  else {
    if (!isnan(temperature) && temperature > adaptiveTempThreshold)
      digitalWrite(RELAY_FAN, LOW);   // ON
    else
      digitalWrite(RELAY_FAN, HIGH);  // OFF
  }

  // Pump control (hysteresis on liquid level)
  if (overridePump)
    digitalWrite(RELAY_PUMP, overridePumpVal ? LOW : HIGH);
  else {
    if (distance != -1) {
      if      (distance < PUMP_OFF_LEVEL) pumpState = true;
      else if (distance > PUMP_ON_LEVEL)  pumpState = false;
    } else pumpState = false;
    digitalWrite(RELAY_PUMP, pumpState ? HIGH : LOW);
  }

  // Buzzer and gas relay
  if (overrideBuzzer) {
    digitalWrite(RELAY_GAS,  overrideBuzzerVal ? LOW  : HIGH);
    digitalWrite(BUZZER_PIN, overrideBuzzerVal ? HIGH : LOW);
  } else {
    bool gas = mq135Detected || mq2Detected;
    digitalWrite(RELAY_GAS,  gas ? LOW  : HIGH);
    digitalWrite(BUZZER_PIN, gas ? HIGH : LOW);
  }
}
```

5. Firebase Data Upload Functions

Two functions handle cloud communication. `postSensorData()` performs an HTTP PUT to overwrite the `/sensors/latest` node with the most recent readings, providing a live snapshot for the dashboard. `postSensorHistory()` performs an HTTP POST to the `/sensors/history` node, appending a new time-stamped entry for trend analysis. Both functions now also include the current `adaptiveTempThreshold` value in the payload, making it possible to correlate fan behaviour against the threshold that was active at any given point in the historical record. Both functions include actuator states and an NTP-derived epoch timestamp (falling back to `millis()` if NTP is unavailable).

Listing 5. postSensorData() and postSensorHistory() – live and historical Firebase uploads with adaptive threshold

```
// ======= POST live sensor data (overwrites latest node) =======
void postSensorData(float temperature, float humidity,
                    bool mq135Detected, bool mq2Detected,
                    long distance) {
  StaticJsonDocument<256> doc;
  if (!isnan(temperature)) doc["temp"] = temperature;
  if (!isnan(humidity))    doc["hum"]  = humidity;
  doc["mq135"] = mq135Detected ? "YES" : "NO";
  doc["mq2"]   = mq2Detected   ? "YES" : "NO";
  if (distance >= 0) doc["distance"] = distance;

  // Include current adaptive threshold for dashboard display
  doc["tempThreshold"] = adaptiveTempThreshold;

  time_t nowSec = time(NULL);
  unsigned long long tsMillis =
    (nowSec > 1600000000)
    ? (unsigned long long)nowSec * 1000ULL
    : (unsigned long long)millis();
  doc["ts"] = tsMillis;

  String body; serializeJson(doc, body);
  String url = String("https://") + firebaseHost + "/sensors/latest.json";
  Serial.print("PUT -> "); Serial.println(url);

  http.begin(client, url);
  http.addHeader("Content-Type", "application/json");
  int httpCode = http.PUT(body);
  Serial.print("HTTP PUT code: "); Serial.println(httpCode);
  http.end();
```

```
}

// ======= POST sensor history (appends to history node) =======
void postSensorHistory(float temperature, float humidity,
                       bool mq135Detected, bool mq2Detected,
                       long distance) {
  StaticJsonDocument<256> doc;
  if (!isnan(temperature)) doc["temp"] = temperature;
  if (!isnan(humidity))    doc["hum"]  = humidity;
  doc["mq135"] = mq135Detected ? "YES" : "NO";
  doc["mq2"]   = mq2Detected   ? "YES" : "NO";
  if (distance >= 0) doc["distance"] = distance;

  // Include actuator states for analytics
  doc["fan"]    = digitalRead(RELAY_FAN)  == LOW  ? 1 : 0;
  doc["pump"]   = digitalRead(RELAY_PUMP) == HIGH ? 1 : 0;
  doc["buzzer"] = digitalRead(BUZZER_PIN) == HIGH ? 1 : 0;

  // Log threshold active at this moment for trend analysis
  doc["tempThreshold"] = adaptiveTempThreshold;

  time_t nowSec = time(NULL);
  unsigned long long tsMillis =
    (nowSec > 1600000000)
    ? (unsigned long long)nowSec * 1000ULL
    : (unsigned long long)millis();
  doc["ts"] = tsMillis;

  String body; serializeJson(doc, body);
  String url = String("https://") + firebaseHost + "/sensors/history.json";

  http.begin(client, url);
  http.addHeader("Content-Type", "application/json");
  int httpCode = http.POST(body);
  Serial.print("HISTORY POST code: "); Serial.println(httpCode);
  http.end();
}
```

6. Cloud Command Polling

The `pollCommands()` function performs an HTTP GET to the `/commands/device1` Firebase node every `POLL_INTERVAL` milliseconds. It parses the returned JSON payload and updates the manual override flags for fan, pump, and buzzer. If a field is absent from the payload, the corresponding override is cleared and automatic control resumes.

Listing 6. pollCommands() – periodic polling of manual override commands from Firebase

```
void pollCommands() {
  String url = String("https://") + firebaseHost
               + "/commands/" + deviceId + ".json";
  http.begin(client, url);
  int code = http.GET();
  if (code == HTTP_CODE_OK) {
    String payload = http.getString();
    Serial.print("Commands JSON: "); Serial.println(payload);

    StaticJsonDocument<256> doc;
    DeserializationError err = deserializeJson(doc, payload);
    if (!err) {
      JsonVariant v;

      v = doc["fan"];
      bool hasFan;  bool fanVal = parseJsonBool(v, hasFan);
      if (hasFan)  { overrideFan    = true; overrideFanVal    = fanVal;  }
      else           overrideFan    = false;

      v = doc["pump"];
      bool hasPump; bool pumpVal = parseJsonBool(v, hasPump);
      if (hasPump) { overridePump   = true; overridePumpVal   = pumpVal; }
      else           overridePump   = false;

      v = doc["buzzer"];
      bool hasBuz;  bool buzVal  = parseJsonBool(v, hasBuz);
      if (hasBuz)  { overrideBuzzer = true; overrideBuzzerVal = buzVal;  }
      else           overrideBuzzer = false;
    } else {
      Serial.print("JSON parse error: "); Serial.println(err.c_str());
    }
  } else {
    Serial.print("Commands GET failed, code="); Serial.println(code);
  }
  http.end();
}
```

7. Adaptive Threshold Fetch Function

The `fetchAdaptiveThreshold()` function is the core of the Adaptive Threshold Engine. It performs an HTTP GET to the `/ml/thresholds/fan` node in Firebase and parses the returned value as a floating-point temperature. Before updating the local variable, a sanity check ensures the received value lies within a physically plausible range of 10 °C to 60 °C. If the value is valid, `adaptiveTempThreshold` is updated immediately and all

subsequent fan decisions use the new value. If the request fails or the value is out of range, the previously stored threshold is retained, providing graceful degradation. The function is called once at boot and then periodically every `THRESHOLD_INTERVAL` (5 s) from the main loop.

Listing 7. fetchAdaptiveThreshold() – polls Firebase for ML-computed fan trigger temperature

```
void fetchAdaptiveThreshold() {
  String url = String("https://") + firebaseHost + thresholdPath + ".json";

  http.begin(client, url);
  int code = http.GET();

  if (code == HTTP_CODE_OK) {
    String payload = http.getString();
    payload.trim();
    payload.replace("\"", ""); // strip quotes if stored as string

    if (payload.length() > 0 && payload != "null") {
      float val = payload.toFloat();

      // Sanity check: reject values outside physically plausible range
      if (val >= 10.0 && val <= 60.0) {
        adaptiveTempThreshold = val;
        Serial.print("Adaptive Threshold Updated: ");
        Serial.println(adaptiveTempThreshold);
      }
    }
  } else {
    Serial.print("Threshold GET failed, code = ");
    Serial.println(code);
  }

  http.end();
}
```

8. Main Loop

The `loop()` function runs continuously. On every iteration it reads all sensors and refreshes the OLED display — which now also shows the current `adaptiveTempThreshold` so operators can see the active fan trigger value locally. The loop then polls Firebase for adaptive threshold updates at `THRESHOLD_INTERVAL` (5 s), polls override commands at `POLL_INTERVAL` (2 s), applies actuator logic, and uploads both live and historical data to Firebase at `POST_INTERVAL` (5 s). A short `delay(200)` at the end prevents busy-looping

while keeping the system highly responsive.

Listing 8. loop() – main execution cycle: sense, display, adaptive threshold update, control, and upload

```
void loop() {
  float humidity    = dht.readHumidity();
  float temperature = dht.readTemperature();
  int   mq135Value  = analogRead(MQ135_PIN);
  bool  mq135Detected = mq135Value > MQ135_THRESHOLD;
  int   mq2Value    = analogRead(MQ2_PIN);
  bool  mq2Detected = mq2Value > MQ2_THRESHOLD;
  long  distance    = readDistanceCM();

  // OLED display update (includes current adaptive threshold)
  display.clearDisplay(); display.setTextSize(1); display.setCursor(0, 0);
  if (!isnan(temperature) && !isnan(humidity)) {
    display.print("T: "); display.print(temperature); display.println("C");
    display.print("H: "); display.print(humidity);    display.println("%");
  } else display.println("DHT Error");
  display.print("Thr: "); display.println(adaptiveTempThreshold, 1);
  display.setCursor(0, 24);
  display.print("MQ135: "); display.println(mq135Detected ? "YES" : "NO");
  display.print("MQ2  : "); display.println(mq2Detected   ? "YES" : "NO");
  display.setTextSize(2); display.setCursor(0, 46);
  display.print("LVL:");
  if (distance != -1) display.print(distance);
  else                display.print("ERR");
  display.display();

  unsigned long now = millis();

  // Refresh adaptive threshold from Firebase every THRESHOLD_INTERVAL
  if (now - lastThresholdMillis > THRESHOLD_INTERVAL) {
    lastThresholdMillis = now;
    fetchAdaptiveThreshold();
  }

  // Poll manual override commands every POLL_INTERVAL
  if (now - lastPollMillis > POLL_INTERVAL) {
    lastPollMillis = now;
    pollCommands();
  }

  applyActuators(temperature, distance, mq135Detected, mq2Detected);

  // Post sensor data every POST_INTERVAL
  if (now - lastPostMillis > POST_INTERVAL) {
```

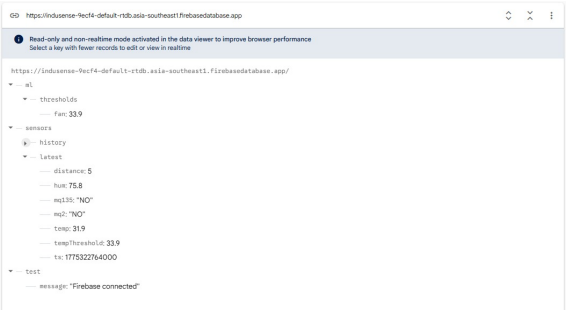
```
    lastPostMillis = now;
    postSensorData(temperature, humidity,
                   mq135Detected, mq2Detected, distance);    // live
    postSensorHistory(temperature, humidity,
                      mq135Detected, mq2Detected, distance); // history
  }

  Serial.print("Temp Thr = "); Serial.println(adaptiveTempThreshold);
  Serial.println("------------------------------------");
  delay(200);
}
```

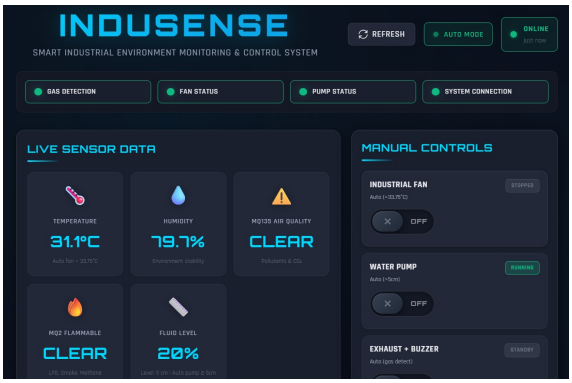
Cloud and Dashboard Implementation

A cloud-based system is implemented using Firebase for data storage and monitoring. The ESP32 uploads the latest sensor readings to the Firebase Realtime Database and also appends historical data for trend analysis. Separate database paths are used for real-time values and history, which makes the dashboard responsive while still preserving time-series records for later observation.

The dashboard visualizes the live industrial condition through a web interface. It displays temperature, humidity, air-quality status, gas leakage indication, and tank level information in real time. In addition, actuator states such as fan, pump, and buzzer status can also be monitored from the dashboard. Remote commands are sent from the dashboard to Firebase, and the ESP32 continuously polls these commands to perform manual override when required. Figure 2 shows the Firebase database node structure alongside the live web dashboard.



(a) Firebase Realtime Database node structure.



(b) Web dashboard showing live sensor readings and actuator controls.

Figure 2. Cloud backend and monitoring dashboard of the InduSense system.

This cloud-dashboard integration provides the following advantages:

- Continuous remote monitoring from any internet-connected device

- Storage of historical sensor data for later analysis
- Real-time visualization of environmental conditions
- Remote manual control of actuators through cloud commands
- Better fault awareness through alarm and status monitoring
- Dynamic fan threshold updates via the `/ml/thresholds/fan` Firebase path, enabling context-aware control without reflashing the firmware

Results

The implemented system successfully achieved real-time monitoring and control of industrial parameters.

- Accurate measurement of temperature, humidity, gas levels, and liquid level
- Reliable actuator control based on sensor thresholds
- Real-time data visualization on OLED and cloud dashboard
- Remote control functionality via cloud commands
- Successful dynamic update of fan threshold via the Adaptive Threshold Engine without firmware reflash

The system demonstrated stable performance in both simulation and hardware implementation. Figure **??** shows the OLED local display during live operation, confirming accurate sensor readings and threshold visibility on-device.

Challenges

During the development of the project, several challenges were encountered:

- Sensor calibration issues, especially for MQ sensors
- Noise and fluctuation in analog sensor readings
- WiFi connectivity instability in certain environments
- Synchronization of real-time tasks using RTOS
- Integration of multiple components without timing conflicts

These challenges were addressed through calibration, filtering techniques, and optimized task scheduling.

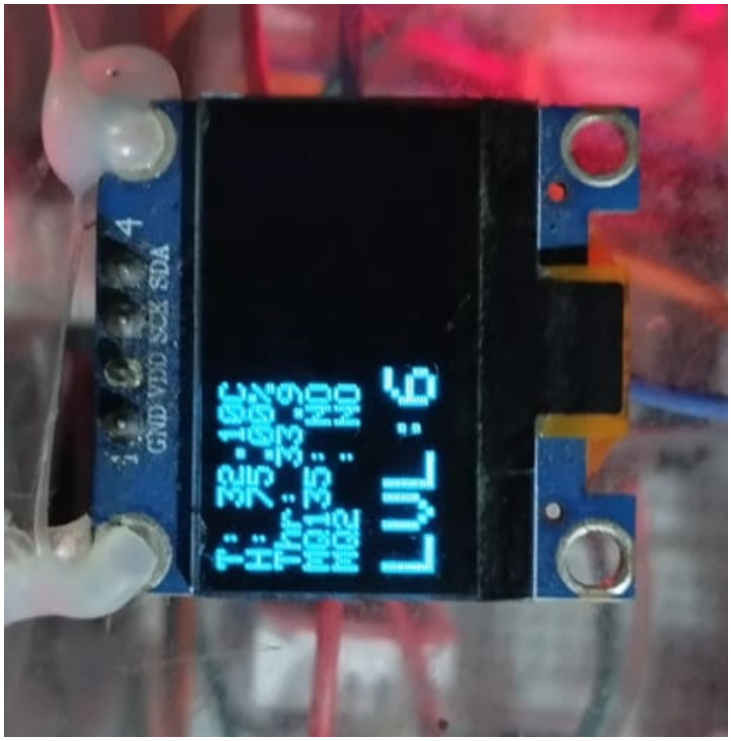


Figure 3. OLED Display Module Showing Real Time Data

Conclusion

The project successfully demonstrates a Smart Industrial Environment Monitoring & Control System using IoT and embedded technologies. It enhances industrial safety by enabling real-time monitoring, automated control, and remote accessibility. The integration of RTOS improves system efficiency and reliability. The Adaptive Threshold Engine further elevates the system by allowing context-aware fan control driven by cloud-updatable parameters, laying the groundwork for future ML-driven automation. This system provides a scalable and cost-effective solution for modern industrial automation.

Future Work

The system can be further improved in the following ways:

- Advanced data analytics and machine learning for anomaly detection
- Implementation of full PLC and SCADA systems
- Enhanced security mechanisms (encryption, authentication)
- Mobile application for user-friendly monitoring
- Expanding the Adaptive Threshold Engine to cover all actuators, not just the fan

References

- ESP32 Technical Reference Manual, Espressif Systems. Available at: https://documentation.es
- Arduino Documentation. Available at: https://docs.arduino.cc/
- Firebase Documentation. Available at: https://firebase.google.com/docs